

EXPERIMENT 4

EXPLORATORY DATA ANALYSIS (EDA) – DATA INSPECTION, FILTERING, STATISTICAL ANALYSIS AND CORRELATION

Aim

To perform data inspection, filtering, grouping, aggregation, descriptive statistical analysis, and correlation analysis using Pandas and visualize the results using Matplotlib.

Algorithm 1

Dataset Inspection

Step 1

Import dataset

Step 2

Display dataset

Step 3

Check dimensions

Step 4

Display data types

Step 5

Generate summary

Program 1

```
: import pandas as pd
```

```
df=pd.read_csv("C:/Users/91754/Downloads/transactions.csv")  
print(df.head())
```

	Transaction ID	Timestamp	Sender Name \
0	4d3db980-46cd-4158-a812-dcb77055d0d2	2024-06-22 04:06:38	Tiya Mall
1	099ee548-2fc1-4811-bf92-559c467ca792	2024-06-19 06:04:49	Mohanlal Bakshi
2	d4c05732-6b1b-4bab-90b9-efe09d252b99	2024-06-04 04:56:09	Kismat Bora
3	e8df92ee-8b04-4133-af5a-5f412180c8ab	2024-06-09 09:56:07	Ayesha Korpai
4	e7d675d3-04f1-419c-a841-7a04662560b7	2024-06-25 08:38:19	Jivin Batta

	Sender UPI ID	Receiver Name	Receiver UPI ID	Amount (INR) \
0	4161803452@okaxis	Mohanlal Golla	7776849307@okybl	3907.34
1	8908837379@okaxis	Mehul Sankaran	7683454560@okaxis	8404.55
2	4633654150@okybl	Diya Goel	2598130823@okicici	941.88
3	7018842771@okhdfcbank	Rhea Kothari	2246623650@okaxis	8926.00
4	1977143985@okybl	Baiju Issac	5245672729@okybl	2800.55

	Status
0	FAILED
1	SUCCESS
2	SUCCESS
3	SUCCESS
4	SUCCESS

Program 2

Shape of Dataset

```
print(df.shape)
```

```
print(df.shape)
```

```
(1000, 8)
```

Program 3

Data Types

```
print(df.dtypes)
```

```
print(df.dtypes)
```

```
Transaction ID    object
Timestamp         object
Sender Name       object
Sender UPI ID     object
Receiver Name     object
Receiver UPI ID   object
Amount (INR)      float64
Status            object
dtype: object
```

Algorithm 2

Filtering Data

Step 1

Specify condition

Step 2

Apply filter

Step 3

Display records

Program 4 - Filter Transactions Greater Than ₹5000

```
high_amount = df[df["Amount (INR)"] > 5000]
print(high_amount)
```

	Transaction ID	Timestamp	\
1	099ee548-2fc1-4811-bf92-559c467ca792	2024-06-19 06:04:49	
3	e8df92ee-8b04-4133-af5a-5f412180c8ab	2024-06-09 09:56:07	
9	718cebcd-cec3-49d5-a24b-d4f6df6ca25f	2024-06-24 09:15:27	
11	35f5cee7-d7a2-42a7-b94f-1d3e54b0bd2e	2024-06-17 17:16:50	
12	71ef7fd4-ea73-432e-9f9d-23ce85d33048	2024-06-28 18:08:24	
..	...	...	
991	a205ecda-b44f-4179-8ff5-da4ead42091e	2024-06-24 19:01:58	
992	7b75d598-1cb7-4e2a-9f9a-6050959978ea	2024-06-09 14:56:46	
994	b98ade62-8c56-4ec1-b265-d69abd3a74c9	2024-06-29 14:20:15	
995	843d87b5-902e-4336-9438-aa76d74e13e4	2024-07-03 23:15:06	
998	27624b55-fd6d-45ce-bbc5-0c2958d48865	2024-06-05 21:57:36	

	Sender Name	Sender UPI ID	Receiver Name	\
1	Mohanlal Bakshi	8908837379@okaxis	Mehul Sankaran	
3	Ayesha Korpall	7018842771@okhdfcbank	Rhea Kothari	
9	Rasha Das	3530841200@okicici	Advik Mahal	
11	Raghav Sridhar	5240349344@okhdfcbank	Shayak Jain	
12	Nirvaan Zachariah	1391606006@okhdfcbank	Ranbir Shukla	
..	...	...	...	
991	Romil Jha	9810396499@okicici	Jivin Sridhar	
992	Alisha Sachdev	1611670234@okaxis	Yuvaan Sengupta	
994	Adah Thakkar	2767535028@oksbi	Yuvraj Wable	
995	Anay Karan	7041132054@okhdfcbank	Dhanush Salvi	
998	Krish Kohli	4638471232@okhdfcbank	Sahil Bail	

Program 5 - Filter Successful Transactions

```
success = df[df["Status"] == "SUCCESS"]
print(success)
```

	Transaction ID	Timestamp	\
1	099ee548-2fc1-4811-bf92-559c467ca792	2024-06-19 06:04:49	
2	d4c05732-6b1b-4bab-90b9-efe09d252b99	2024-06-04 04:56:09	
3	e8df92ee-8b04-4133-af5a-5f412180c8ab	2024-06-09 09:56:07	
4	e7d675d3-04f1-419c-a841-7a04662560b7	2024-06-25 08:38:19	
5	865cfff6-e345-4bae-a4ed-2c35c3a67fc7	2024-06-17 11:24:02	
..	...	...	
990	2f5bb403-603f-48f0-a289-d97949768e8b	2024-07-01 07:59:56	
991	a205ecda-b44f-4179-8ff5-da4ead42091e	2024-06-24 19:01:58	
994	b98ade62-8c56-4ec1-b265-d69abd3a74c9	2024-06-29 14:20:15	
995	843d87b5-902e-4336-9438-aa76d74e13e4	2024-07-03 23:15:06	
999	b792bf64-2047-4435-aac1-27ee95b134d6	2024-06-21 15:52:40	

	Sender Name	Sender UPI ID	Receiver Name	\
1	Mohanlal Bakshi	8908837379@okaxis	Mehul Sankaran	
2	Kismat Bora	4633654150@okybl	Diya Goel	
3	Ayesha Korpall	7018842771@okhdfcbank	Rhea Kothari	
4	Jivin Batta	1977143985@okybl	Baiju Issac	
5	Kismat Rama	5438757524@okicici	Pihu Walia	
..	...	...	...	
990	Manikya Barad	4470551109@okicici	Faiyaz Mall	
991	Romil Jha	9810396499@okicici	Jivin Sridhar	
994	Adah Thakkar	2767535028@oksbi	Yuvraj Wable	
995	Anay Karan	7041132054@okhdfcbank	Dhanush Salvi	
999	Diya Trivedi	0385561234@oksbi	Dhanuk Mane	

Algorithm 3

Sorting Data

Step 1

Select column

Step 2

Apply sort_values()

Step 3

Display sorted result

Program 6

Sort transaction by amount-descending

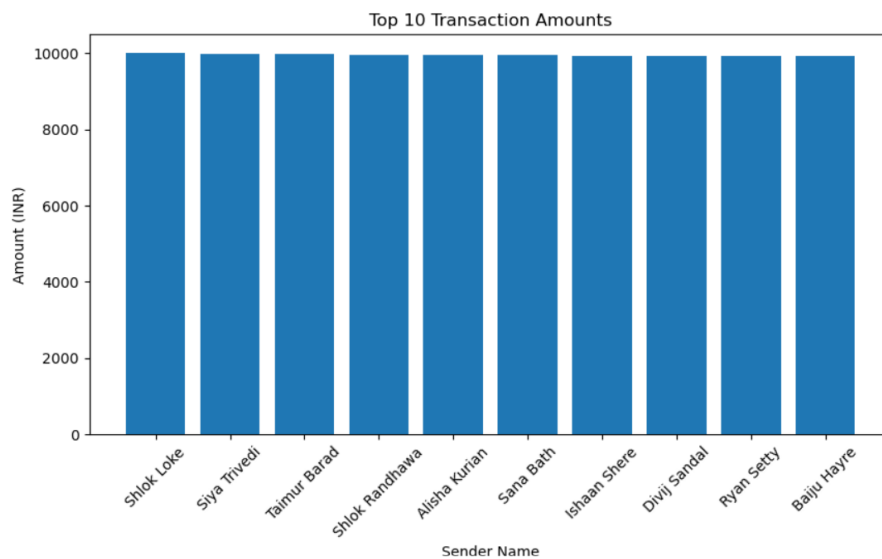
```
sorted_df = df.sort_values(  
    by="Amount (INR)",  
    ascending=False  
)  
print(sorted_df)
```

	Transaction ID	Timestamp \
484	7955e217-102f-41e1-985f-4a3dfb1abae	2024-07-01 01:20:13
442	81ae109e-2d5b-40c3-8308-c5000cf8b519	2024-06-27 03:14:57
415	bb92963b-f1fc-4b8b-83b1-6c804d38df80	2024-06-20 00:55:03
451	6b8bd593-9e2d-46f8-9fc7-82e79faeed6c	2024-06-07 17:55:00
561	6140dc16-7d90-4ad0-b306-0d13d110f705	2024-06-20 22:31:26
..	...	...
333	c5e2904f-018b-49fe-8f10-9689af421626	2024-06-21 16:56:31
500	fad27c17-8e1b-4711-8b02-7eaf144a1cee	2024-06-21 03:29:15
546	2176ff0a-f24a-42bb-acb9-2af254c156bd	2024-06-09 10:05:58
453	f1b151a2-f553-423a-ac39-8d1ae6cf439d	2024-06-20 08:09:34
282	62660621-a015-4ad9-9fca-6dcb0c6c357e	2024-06-30 23:33:27

	Sender Name	Sender UPI ID	Receiver Name
484	Shlok Loke	8347576235@okicici	Devansh Wagle
442	Siya Trivedi	8562122542@okybl	Mishti Chaudry
415	Taimur Barad	3173202042@okybl	Tarini Shukla
451	Shlok Randhawa	3676327128@okybl	Samar Sawhney
561	Alisha Kurian	1176434262@oksbi	Tara Dayal
..	...	...	...
333	Indranil Yohannan	8482699080@okybl	Siya Sura
500	Vritika Madan	8421651232@okhdfcbank	Shlok Sachdev
546	Taran Sathe	8349539675@okybl	Indrajit Kar

Visualization - Transaction Ranking

```
import matplotlib.pyplot as plt
plt.figure(figsize=(10,5))
top10 = sorted_df.head(10)
plt.bar(top10["Sender Name"], top10["Amount (INR)"])
plt.title("Top 10 Transaction Amounts")
plt.xlabel("Sender Name")
plt.ylabel("Amount (INR)")
plt.xticks(rotation=45)
plt.show()
```



Algorithm 4

Statistical Analysis

Step 1

Select numerical column

Step 2

Calculate Mean

Step 3

Calculate Median

Step 4

Calculate Mode

Step 5

Calculate Standard Deviation

Program 7

```

: print("Mean =", df["Amount (INR)"].mean())
  print("Median =", df["Amount (INR)"].median())
  print("Mode =")
  print(df["Amount (INR)"].mode())
  print("Standard Deviation =", df["Amount (INR)"].std())

Mean = 4999.02293
Median = 4951.4349999999995
Mode =
0      4779.28
Name: Amount (INR), dtype: float64
Standard Deviation = 2873.484533761384

```

Program 8- Descriptive Statistics

```
print(df.describe())
```

```
print(df.describe())
```

	Amount (INR)
count	1000.000000
mean	4999.022930
std	2873.484534
min	28.520000
25%	2521.620000
50%	4951.435000
75%	7315.835000
max	9993.060000

Algorithm 5

Grouping and Aggregation

Step 1

Select Status Column

Step 2

Apply groupby()

Step 3

Calculate Average Amount

Step 4

Display Result

Program 9

Average Transaction Amount By Status

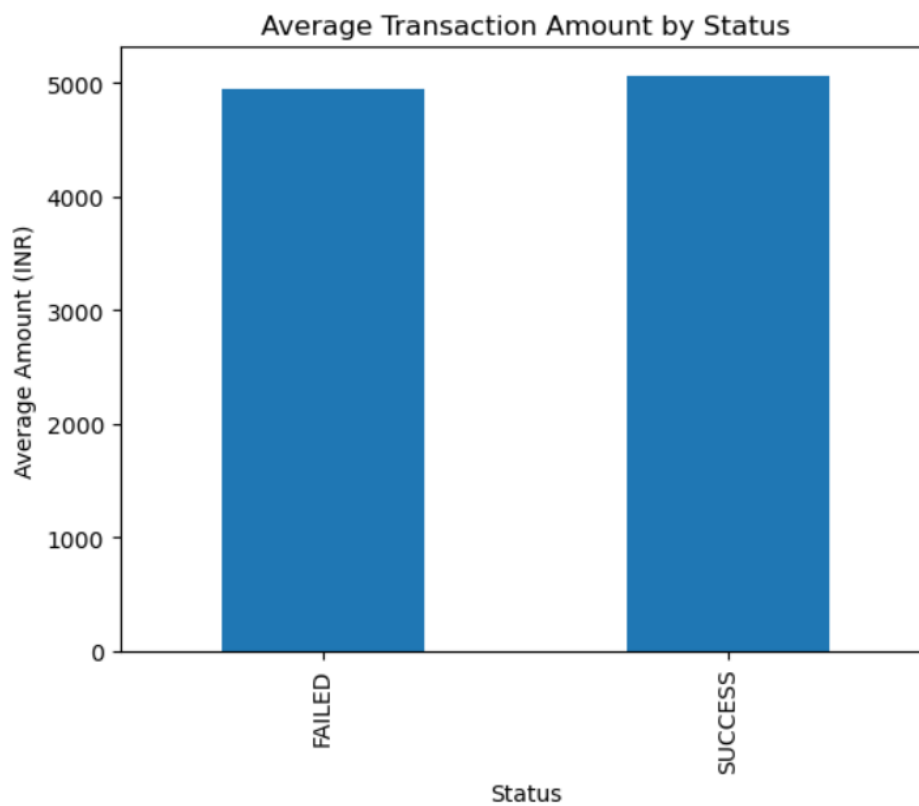
```
status_avg = df.groupby(  
    "Status"  
)["Amount (INR)"].mean()
```

```
print(status_avg)
```

```
Status  
FAILED      4938.685221  
SUCCESS     5058.879861  
Name: Amount (INR), dtype: float64
```

Visualization

```
status_avg.plot(kind="bar")  
plt.title("Average Transaction Amount by Status")  
plt.xlabel("Status")  
plt.ylabel("Average Amount (INR)")  
plt.show()
```



Algorithm 6

Correlation Analysis

Step 1

Select numerical columns

Step 2

Apply corr()

Step 3

Interpret correlation coefficient

Program 10

```
#convert timestamp to date/time feature
df["Timestamp"] = pd.to_datetime(df["Timestamp"])
df["Hour"] = df["Timestamp"].dt.hour

#calculate correlation
corr = df[[
    "Amount (INR)",
    "Hour"
]].corr()
print(corr)
```

	Amount (INR)	Hour
Amount (INR)	1.000000	0.021028
Hour	0.021028	1.000000

Result

Thus, data inspection, filtering, sorting, descriptive statistical analysis, grouping, aggregation, and correlation analysis were successfully performed using Pandas. The analytical results were visualized and interpreted successfully.